ChatGPT

EduDocu Manager 요구사항 (Google Sheets/ AppSheet 기반)

개정일: 2026년 2월 27일
개정 사유: 기존 문서는 로컬 데이터베이스(Electron + SQLite)를 전제로 작성되었으나, 이 문서는 **Google Sheets를 데이터베이스로 사용하고 AppSheet/Apps Script를 활용**하는 방향으로 수정하였다. Google Sheets는 클라우드 기반으로 접근성과 협업이 뛰어나며, AppSheet는 코드 없이도 직관적인 UI를 생성하고 Google Workspace 데이터를 활용할 수 있다 [1].

---

1. 시스템 개요

- **앱 이름**: EduDocu Manager (에듀도큐 매니저)
- **목적**: 방과후학교 강사가 학생·교구·수납 데이터를 체계적으로 관리하고 학교별 행정 서류를 자동 생성하는 업무 도구를 **Google Sheets와 AppSheet**를 통해 구현한다. AppSheet는 Google Sheets에 저장된 데이터를 기반으로 모바일·웹에서 바로 사용할 수 있는 UI를 제공하고, Apps Script는 복잡한 로직·자동화(예: 서류 생성, 공휴일 계산) 구현에 사용된다 [2].
- **핵심 원칙**:
- **Google Sheets 기반 영구 저장**: 모든 데이터는 Google Sheets 스프레드시트에 영구 저장된다. Google Sheets는 클라우드 기반으로 데이터가 안전하게 저장되고 접근·편집 권한을 쉽게 제어할 수 있다 [1].
- **CRUD + 검색 가능**: AppSheet를 통해 각 시트의 데이터를 추가·조회·수정·삭제하며, 검색 및 필터 기능을 제공한다.
- **학교별 양식 대응**: 학교별로 다른 계획서·출석부 양식을 Google Sheets 템플릿과 Apps Script를 이용해 맞춤 적용한다.
- **엑셀 자동 생성 및 분리 저장**: 서류는 Google Sheets에서 자동 생성되며, 필요 시 Apps Script를 이용해 시트를 Excel 파일(.xlsx)로 내보낼 수 있다 [3].

---

2. 데이터 관리 요구사항

2.1 저장 방식 (Google Sheets 기반)

- **Google Sheets 기반 스토리지**: 모든 엔티티(학교, 학생, 커리큘럼, 출석, 수납)는 하나의 Google Sheets 파일 내에서 개별 시트로 분리해 저장한다. Google Sheets는 클라우드에 저장되므로 데이터가 안전하고 여러 사용자가 동시에 협업할 수 있다 [1].
- **고유 식별자 활용**: 각 시트에는 `id` 열을 두어 고유 식별자를 부여한다. AppSheet는 이 열을 키로 사용해 레코드 간 관계를 연결한다.
- **권한 관리**: Google Sheets의 공유 기능으로 데이터 접근 권한을 세분화하며, 학생 개인정보가 포함된 시트는 최소 권한 원칙에 따라 제한된 사용자만 편집할 수 있도록 한다.
- **오프라인 접근**: Google Sheets 오프라인 편집 기능을 활용해 인터넷 연결이 없어도 데이터를 열람·수정할 수 있지만, 대규모 데이터 변경 시 동기화 지연을 고려한다.

2.2 CRUD 기능

모든 데이터 항목에 대해 AppSheet를 통한 CRUD 기능을 제공한다. AppSheet는 시트에 연결된 폼과 테이블을 자동 생성하며 기본 검색·필터 기능을 포함한다. 주요 기능은 다음과 같다.

- **추가 (Create)**: 폼 기반 입력 화면에서 새 레코드를 Google Sheets 시트에 추가한다.
- **조회 (Read)**: 테이블·카드·갤러리 뷰를 통해 데이터 목록과 상세 정보를 조회한다. 관계형 데이터는 참조(Ref) 컬럼을 설정하여 관련 레코드를 쉽게 찾아볼 수 있다.
- **수정 (Update)**: 각 레코드의 상세 보기 화면에서 필드를 수정하면 Google Sheets 데이터가 업데이트된다.
- **삭제 (Delete)**: 선택한 레코드를 삭제할 수 있으며, AppSheet의 역할 기반 권한을 통해 삭제 권한을 제한할 수 있다.
- **검색·필터 (Search)**: AppSheet는 기본 검색 및 필터 기능을 제공한다. 추가적으로 Apps Script나 Sheets API를 통한 고급 검색 로직을 구현할 수 있다 [4] .

2.3 데이터 구조

각 데이터는 Google Sheets 내 개별 시트로 관리된다. 예를 들어 `schools`, `students`, `curricula`, `attendance`, `payments`와 같은 시트를 생성하고, 각 시트는 아래와 같이 열을 가진다. AppSheet에서는 이러한 시트와 열을 불러와 테이블·폼·참조 관계를 자동 구성한다.

학교 데이터 (schools 시트)

열 이름	설명
`id` (키)	고유 ID (자동 생성)
`name`	학교명
`fee_per_student`	수강료 단가
`kit_fee`	교구비 단가
`class_days`	수업 요일 목록 (예: 화,목)
`class_time_A`, `class_time_B`, `class_time_C`	반별 수업 시작시간·종료시간
`template_monthly_plan` ~ `template_kit_list`	학교별 양식 파일(Drive 링크)

학생 데이터 (students 시트)

열 이름	설명
`id` (키)	고유 ID
`school_id`	참조: schools.id
`class_name`	반(A, B 등)
`name`	학생 이름
`grade`	학년
`number`	반 내 번호
`parent_contact`	학부모 연락처(숫자만 저장)

열 이름	설명
`kit_status`	교구비 상태 (새교구/기존교구/미사용)
`enrolled_at`	등록일

커리큘럼 데이터 (curricula 시트)

열 이름	설명
`id` (키)	고유 ID
`school_id`	참조: schools.id
`session_no`	회차 번호
`title`	수업 주제
`content`	상세 내용
`kit_type`	교구 유형(항공, 안전망드론 등)
`plan_file_url`	계획서 파일(Drive 링크)
`guide_file_url`	지도안 파일(Drive 링크)
`order`	Drag & Drop 순서를 저장하는 정수

출석 데이터 (attendance 시트)

열 이름	설명
`id` (키)	고유 ID
`student_id`	참조: students.id
`date`	수업 날짜
`status`	출석 상태(출석/결석/보강)
`memo`	특이사항 메모

수납 데이터 (payments 시트)

열 이름	설명
`id` (키)	고유 ID
`student_id`	참조: students.id
`year`	연도
`month`	월
`tuition_paid`	수강료 납부 여부(BOOLEAN)

열 이름	설명
`kit_paid`	교구비 납부 여부(BOOLEAN)
`payment_date`	실제 납부일

각 시트의 관계를 AppSheet에서 `Ref` 타입으로 설정하면 학생 ↔ 학교, 커리큘럼 ↔ 학교 등의 관계를 쉽게 표현할 수 있다.

---

3. 자동 계산 요구사항

3.1 수업 날짜 계산

- **대한민국 공휴일 API 연동**: Apps Script를 이용해 공공데이터포털의 특일 정보 API를 호출하고, 결과를 `holidays` 시트에 저장한다. 공휴일 목록은 캐시되어 반복 호출을 줄인다. 불가피하게 API 호출에 실패한 경우 수동 입력 UI를 제공한다 [2].
- **법정 공휴일 자동 제외 및 대체휴무 반영**: Apps Script에서 지정 월의 수업 요일(예: 화·목)에 해당하는 모든 날짜를 생성한 뒤, `holidays` 시트에 있는 공휴일을 제외하고 대체휴무일도 반영한다. 계산된 결과는 `class_dates` 시트나 가상 컬럼으로 저장된다.
- **오프라인 대체**: API가 제공되지 않을 때를 대비하여 Google Sheets에 미리 기본 공휴일 목록을 제공하고 필요 시 갱신한다.

3.2 커리큘럼 시퀀싱

- **회차 순서 자동 매핑**: 커리큘럼 목록(`curricula` 시트)은 `order` 컬럼을 기준으로 정렬되며, AppSheet의 Drag & Drop 기능을 사용해 순서를 변경한다. 변경 시 Apps Script 또는 AppSheet의 `Automation` 기능을 통해 `session_no`와 `order`를 자동 업데이트한다.
- **Gemini 분석 시 순서 유지**: Google AI Studio를 활용해 텍스트 계획서를 분석하여 회차별 주제·내용을 추출할 때, 원본 순서를 `order` 컬럼에 그대로 반영한다.

3.3 비용 계산

- **강사료 자동 계산**: 학생 수와 수강료, 수업 횟수를 곱하여 월별 강사료를 계산한다. 해당 계산은 Google Sheets 함수 또는 Apps Script를 통해 자동화한다.
- **교구비 자동 계산**: `kit_status`에 따라 교구비를 부과한다. `kit_status='새교구'`이면 학교의 `kit_fee`를 적용하고, `기존교구`면 10,000원을 적용하며, `미사용`이면 0원으로 설정한다. 계산 결과는 `payments` 시트에 저장한다.

---

4. 파일 관리 요구사항

4.1 업로드 기능

- **파일 저장 위치**: Google Sheets는 파일 자체를 저장할 수 없으므로, 샘플 서류 양식(Excel·PDF)과 커리큘럼 계획서/지도안 파일은 **Google Drive**에 업로드한다. 업로드한 파일의 URL을 학교나 커리큘럼 시트의 `template_…` 또는 `plan_file_url` / `guide_file_url` 컬럼에 저장한다.
- **지원 형식**: Excel(.xlsx), PDF, 이미지(JPG/PNG) 등. Drive에 업로드된 파일은 필요한 경우 Apps Script를 통해 읽거나 내보낼 수 있다.

4.2 서류 생성 기능

생성 대상 서류는 기존과 동일하지만 생성 방식이 변경된다. Apps Script를 활용하여 Google Sheets에서 서류용 시트를 생성하고 필요하면 Excel 형식으로 내보낼 수 있다 [3].

- **월간계획서**: `class_dates`와 `curricula` 정보를 조합하여 `YYYY-MM 월간계획서` 시트 생성.
- **수강생명단**: `students` 데이터를 기준으로 `YYYY-MM 수강생명단` 시트 생성(학생명 가나다순 + 교구비 상태).
- **출석부**: `students` × `class_dates` 행렬 형태의 `YYYY-MM 출석부` 시트 생성.
- **활동일지**: 월간계획서 내용을 상세히 기록한 `YYYY-MM 활동일지` 시트 생성.
- **강사료 청구서**: 강사료 계산 결과를 요약한 `YYYY-MM 강사료청구서` 시트 생성.
- **교구비 명단**: 교구비 납부 여부를 정리한 `YYYY-MM 교구비명단` 시트 생성.

각 시트는 Google Sheets 내에서 작성되며 Apps Script를 통해 템플릿 적용, 셀 병합, 서식 설정 등을 자동화한다. 필요 시 `export?format=xlsx` 형식의 URL을 이용해 사용자가 Excel 파일로 다운로드할 수 있다 [3].

4.3 파일 구조 요구사항

- **서류 시트 구성**: 생성된 서류는 Google Sheets의 **스프레드시트 내 여러 시트**로 구성하며, 월별·반별로 시트를 분리한다. 시트명 규칙은 `MM월-A`, `MM월-B` 등으로 통일한다.
- **학교 양식 템플릿 적용**: 각 학교에서 제공한 엑셀 템플릿 파일을 Google Drive에 업로드하여 템플릿 시트로 변환한 후, Apps Script를 통해 해당 시트를 복제하여 서류 생성 시 적용한다.

---

5. UI 요구사항

5.1 대시보드

AppSheet의 **Dashboard** 뷰를 사용해 여러 차트·테이블을 한 화면에 표시한다.

- **학교 목록**: `schools` 시트에서 학교별 카드를 보여주고, 각각의 학생 수·반 수를 요약한다.
- **학생 수 통계**: 차트 뷰를 통해 학교별·반별 학생 수를 시각화한다.
- **서류 생성 현황**: `DocumentRecord` 시트(서류 생성 이력)에서 최근 생성된 서류를 표시하고, 미생성 서류에 대한 알림을 제공한다.

5.2 데이터 입력 화면

AppSheet의 **Form** 및 **Table** 뷰를 활용하여 다음 항목을 관리한다.

- **학교 관리**: `schools` 시트를 편집하는 Form/Table 뷰. 학교 추가·수정·삭제, 수업 요일 및 시간 설정, 양식 템플릿 URL 등록 등을 지원한다.
- **학생 관리**: `students` 시트를 편집하는 Deck/Table 뷰. 학교 선택 후 반별로 학생 목록을 보여주고, 학생 추가·수정·삭제를 수행한다. 연락처 입력 시 숫자만 허용하고 출력은 `000-0000-0000` 형식으로 포맷팅한다.
- **커리큘럼 관리**: `curricula` 시트를 스프레드시트 형태로 보여주며, Drag & Drop으로 `order`를 변경할 수 있는 인터페이스를 제공한다. 계획서/지도안 URL 업로드도 여기서 진행한다.
- **출석 관리**: `attendance` 시트를 편집하는 Calendar/Table 뷰로, 날짜별 출석 상태와 메모를 기록한다.
- **수납 관리**: `payments` 시트를 편집하는 Table 뷰로, 월별 납부 현황을 관리한다.

5.3 문서 생성 화면

문서 생성은 AppSheet의 **Action** 및 **Automation** 기능을 활용하여 제공한다.

- **학교 선택**: 드롭다운을 통해 `schools` 시트에서 학교를 선택한다.
- **월 선택**: Date Picker를 통해 생성할 연도와 월을 선택한다.
- **생성 서류 체크박스**: 생성할 서류 종류(월간계획서, 출석부, 청구서 등)를 선택한다.
- **미리보기**: 선택한 서류를 Google Sheets의 미리보기 링크로 열어 확인하거나, AppSheet 안에서 테이블로 간단히 미리 본다.
- **엑셀 다운로드**: Apps Script에서 생성된 서류 시트들을 Excel로 변환하고, 사용자에게 다운로드 링크를 제공한다.

---

6. AI 기능 요구사항

- **Gemini API 계획서 분석**: Google AI Studio에서 제공하는 Gemini API를 사용하여 텍스트 계획서를 분석하고 `curricula` 시트에 회차별 주제·내용·교구 유형을 자동 추출한다. 분석된 데이터는 순서를 유지하며 `order` 컬럼에 저장한다.
- **AI 기반 음성/텍스트 입력 지원**: AppSheet의 AI 생성 기능을 사용하여 사용자가 자연어로 수업 내용이나 특이사항을 입력하면 자동으로 필드에 구조화된 데이터로 변환해 저장한다.
- **순서 유지 처리**: AI 분석 결과의 원본 순서를 보존하기 위해 각 회차에 고유 `order` 번호를 할당하고, Drag & Drop으로 수정될 때도 AI가 제안한 논리적 순서를 참고할 수 있도록 한다.

---

7. 보안 요구사항

- **개인정보 보호**: 학생 이름, 연락처 등 개인정보는 Google Sheets의 접근 권한을 통해 보호한다. 시트를 공유할 때 보기 권한과 편집 권한을 구분하고, 민감한 시트는 특정 사용자에게만 공유한다.
- **Google OAuth 인증**: AppSheet 앱 사용자는 자신의 Google 계정으로 로그인하며, 앱은 Sheets·Drive API에 접근할 수 있는 최소한의 OAuth 범위를 요청한다.
- **API 키 보호**: 공휴일 API 키와 Gemini API 키는 Apps Script의 Properties Service 또는 Google Cloud Secret Manager에 저장하여 코드에 노출되지 않도록 한다.
- **파일·코드 관리**: Google Sheets와 Apps Script는 클라우드에서 관리되므로 로컬 DB 파일은 존재하지 않는다. GitHub에는 Apps Script 코드의 사본을 업로드할 수 있지만 민감한 정보는 `.clasp.json` 등의 파일에서 제외한다.

---

8. 확장 요구사항

- **알림 기능**: AppSheet의 Notification 및 Automation 기능을 활용해 서류 제출 마감일, 수업 시작 전 알림(D-3, D-1 등)을 푸시 알림 또는 이메일로 전송한다.
- **사진 첨부 활동일지**: Google Drive와 연동하여 활동일지에 사진을 업로드하고, 업로드된 이미지의 Drive URL을 `activity_photos` 시트에 저장한 뒤 Apps Script를 통해 서류에 삽입한다.
- **수익 통계 대시보드**: Google Data Studio(현재 Looker Studio)와 연동하거나 AppSheet의 차트 뷰를 이용해 월별 수강료·교구비 수익을 시각화한다.
- **학교별 자동 제출 패키지 생성**: Apps Script를 사용하여 월별로 생성된 서류를 묶어 ZIP 파일로 만들고 Google Drive에 저장하며, 해당 링크를 학교별 담당자에게 자동 공유한다.

---

9. GitHub 관리 요구사항

- **.gitignore 설정**: Apps Script 프로젝트를 `clasp`로 관리하는 경우 `.clasp.json`과 `appsscript.json`을 제외하고, API 키 등 민감한 정보가 저장된 파일은 `.gitignore`에 추가한다.
- **README 작성**: Google Sheets 구조, AppSheet 설정 방법, Apps Script 배포 방법 등을 문서화한다.
- **민감 정보 제외**: API 키, OAuth 자격증명, 개인용 스프레드시트 ID 등은 GitHub에 포함하지 않는다.

---

10. Build 적용 원칙

- **Google AI Studio·AppSheet 우선 적용**: 자연어 기반 설계 도구(Google AI Studio Build)를 사용해 AppSheet 앱을 자동 생성하고, 생성된 구성요소를 필요한대로 수정한다. Build는 데이터 구조와 UI를 자연어 설명으로부터 생성할 수 있으며, 이를 바탕으로 신속한 프로토타이핑을 진행한다.
- **최신 모델 및 API 사용**: Gemini API 등 최신 AI 기능을 우선적으로 적용하여 커리큘럼 분석과 자연어 입력 처리를 자동화한다.
- **데이터·UI 구조 최적화**: Build가 자동으로 생성한 Google Sheets 구조와 AppSheet 뷰를 프로젝트 특성에 맞게 리팩토링한다. 예를 들어 통합 시트가 너무 커지면 분리하거나, 시트 간 관계를 명확히 하기 위해 키/Ref 컬럼을 추가한다.
- **오픈 확장성**: 필요한 경우 Sheets API, Apps Script, Looker Studio 등 다른 Google Workspace 도구와 연동해 기능을 확장한다.

(End of Requirements)

---

1 Google Sheets as a Database: Pros & Cons
https://stackby.com/blog/google-sheets-database/

2 When to use AppSheet or Apps Script in Google Workspace | Google Workspace Blog
https://workspace.google.com/blog/developers-practitioners/when-to-use-appsheet-or-apps-script-in-google-workspace

3 Comprehensive guide: Export Google Sheets as PDF, Excel, CSV, or other formats using Apps Script
https://spreadsheet.dev/comprehensive-guide-export-google-sheets-to-pdf-excel-csv-apps-script

4 Google Sheets API Overview | Google for Developers
https://developers.google.com/workspace/sheets/api/guides/concepts